Robotics Lab 2025/2026

Homework 1

Bring up your robot

Liccardo Antonio P38/136

GitHub Profile: https://github.com/LiccardoAntonio

For the purpose of streamlining the development of the code, the following adjustements and tools were used:

- Terminator was the terminal chosen, as the use of a custom layout can bring up and connect to the Docker ROS2 environment quickly;
- VSCode was started inside of the directory connected to the Docker `/src` directory of the workspace. The use of VSCode ROS2 extensions helped in correcting the syntax and keeping the workspace organized.

Bring up your robot

1. To start development, the provided `armando_description` package was cloned into the `/src` directory.

   (a) Within the package, the `/launch` directory was created, and within that the `armando_display.launch.py`. The launch script is based on the `urdf_display.launch.py` from the `ros2_urdf` package provided by the course's organization (https://github.com/RoboticsLab2025/ros2_urdf/).
   The code deals with declaring lauch arguments relative to the startup of the `joint_state_publisher_gui` (which is optional), the path to the RViz2 configuration file and the path to the URDF file.
   The `urdf_launch` (https://github.com/ros/urdf_launch) package has been installed in the `/src` directory to be used in this launch file, for the purpose of loading the URDF as a ROS2 Parameter and then launching the `robot_state_publisher`. Finally, the `Node` objects are declared for the `joint_state_publisher`, the `joint_state_publisher_gui` and the `rviz2` and the `LaunchDescription` is created and returned.
   The following lines of code have been added to the package manifest to express the dependencies:

```
<depend>joint_state_publisher</depend>
<depend>joint_state_publisher_gui</depend>
<depend>robot_state_publisher</depend>
<depend>rviz</depend>
```

   The following command has been added to the `CMakeList.txt` in order to build the directories within the package:

```
install(DIRECTORY
launch urdf meshes
DESTINATION share/${PROJECT_NAME}/
)
```

   With respect to the original URDF file, paths to the meshes of the armando manipulator in the `<link>` objects have been modified, since the original paths referred to the package name as `arm_description` instead of `armando_description`. Later in the development process the paths to the meshes have been modified to be absolute in order to to avoid conflicts in the relative pathing syntax with Gazebo.

   (b) After building the workspace, sourcing and launching with:

```
$ros2 launch armando_description armando_display.launch.py
```

the RViz2 environment and joint state publisher gui are brought up. On the left-side panel the `RobotModel` panel has been added. Under the Global Option, the Fixed Frame has been set to `base_link`, under RobotModel the Description Source has been set to `Topic`, and the `/robot_description` topic has been selected as the Description Topic. In figure 1 is show the environment's appearance after implementing these changes.

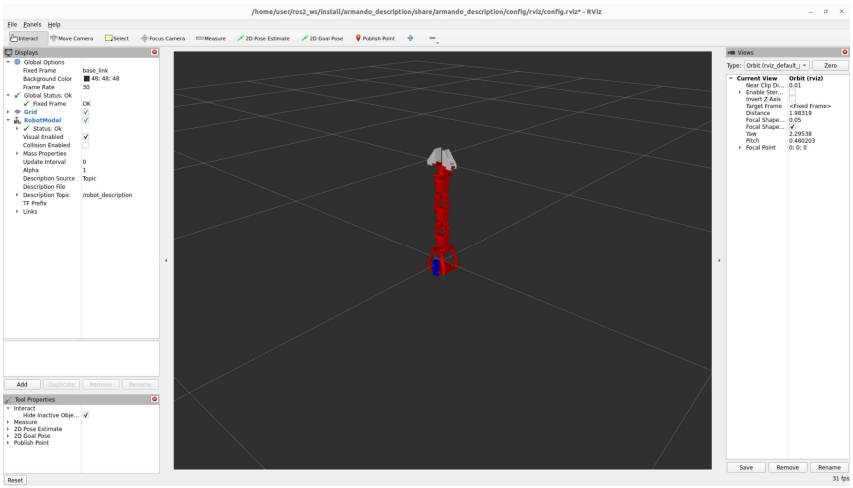

Figure 1: Rviz2 environment after configuring the display settings

Finally, these settings have been saved as `config.rviz` in the `/config/rviz` directory, included in the `armando_description` package. The `/config` directory has then been added to the `CMakeList.txt` as follows:

```
install(DIRECTORY
launch urdf meshes config
DESTINATION share/${PROJECT_NAME}/
)
```

(c) For the last point of the first part, the `<mesh>` tag under the `<collision>` tag in the URDF has been subsituted with `<box>` tag as primitive geometry elements. These elements were iteratively adjusted in size and position by displaying them in Rviz2. In the Rviz2 environment the measure tool aided this process. As an implementative choice, the boxes boundaries are never smaller than the visual meshes to keep a more conservative margin in simulation. This is better visualized by setting the alpha value to 0.5 and enabling collisions, as shown in Figure 2.

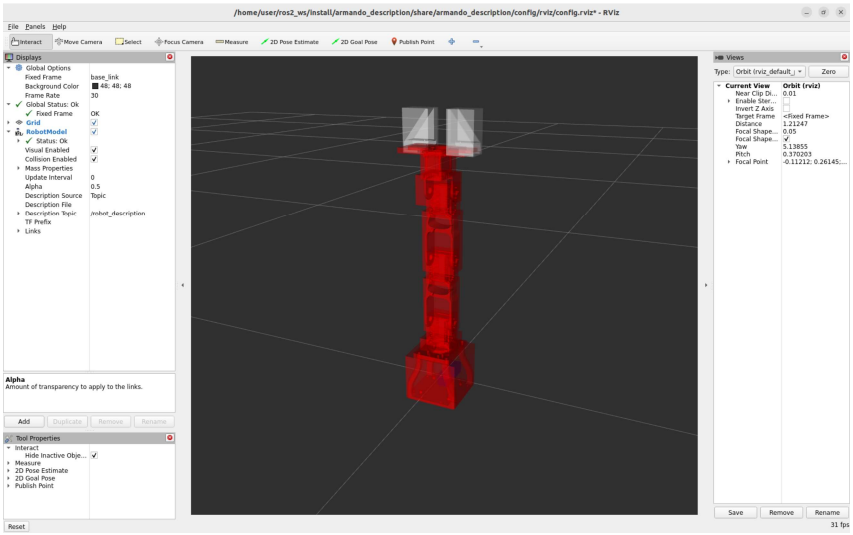

Figure 2: Collision boxes

2. The second project specification is about the implementation of hardware interfaces and controllers, and simulation of the Armando manipulator in the Gazebo environment.

   (a) The `armando_gazebo` package was created by .

   ```
   ros2 pkg create --build-type ament_cmake --license Apache-2.0 armando_gazebo
   ```

   The directory `/launch`, and within it the `armando_world.launch.py` launch file, were also created. As suggested by the specification, the launch file was based on `gazebo.launch.py` from the `ros2_urdf` package (https://github.com/RoboticsLab2025/ros2_urdf). Initially the URDF was parametrized and loaded in the `robot_state_publisher` in the same way as it was loaded in the `armando_display.launch` launch file, then following implementative specifications defined in the point 2.b, this was shortly changed, so it will be better discussed in the next point.
   Within the launch file, the `IncludeLaunchDescription` function is used to call the `gz_sim.launch.py` in order to bring up the gazebo empty world simulation:

   ```
   empty_world_launch = IncludeLaunchDescription(
       PathJoinSubstitution([FindPackageShare('ros_gz_sim'), 'launch', 'gz_sim.
       launch.py']),
       launch_arguments={
           'gui': LaunchConfiguration('gui'),
           'pause': 'true',
           'gz_args': ['-r ', 'empty.sdf'],
       }.items(),
   ```

   The simulation is then populated by the `create` node, that uses the `/robot_description` topic as parameter to load the Armando manipulator:

   ```
   urdf_spawner_node = Node(
       package='ros_gz_sim',
       executable='create',
   ```

```
    name='urdf_spawner',
    arguments=['-topic', '/robot_description', '-entity', 'robot', '-z', '
0.5', '-unpause'],
    output='screen',
)
```

(b) To fulfill the specification 2.b, the URDF file was updated to a .xacro file, under the name `arm.urdf.xacro`. With the purpose of generating and loading the URDF as parameter in the `robot_state_publisher` node, the following routine (taken from the `iiwa.launch.py` launch file, in the `ros2_sensors_and_actuators` package at https://github.com/RoboticsLab2025/ros2_sensors_and_actuators) has been implemented:

```
xacro_armando = os.path.join(get_package_share_directory('
armando_description'), "urdf", "arm.urdf.xacro")

robot_description_armando_xacro = {"robot_description": Command(['xacro ',
xacro_armando])}

robot_state_publisher_node = Node(
    package="robot_state_publisher",
    executable="robot_state_publisher",
    output="both",
    parameters=[robot_description_armando_xacro,
                {"use_sim_time": True},
        ],
)
)
```

This routine deals with:

- finding the path through the `os.path.join` function;
- generating the URDF from the xacro file through the `Command` function;
- passing the URDF as a parameter to the Node element relative to the `robot_state_publisher` node.

In the `arm.urdf.xacro` file the `<robot>` tag has been updated to the following, in order to be able to use macros in the file.

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="arm">
```

The `<ros2_control name="PostionJointInterface" type="system">` tag has been added to define the hardware and its interfaces for the Armando manipulator:

```
<ros2_control name="PostionJointInterface" type="system">

  <hardware>
    <plugin>ign_ros2_control/IgnitionSystem</plugin>
  </hardware>

  <xacro:armando_hardware_interface joint_name="j0"/>
```

```
    <xacro:armando_hardware_interface joint_name="j1"/>
    <xacro:armando_hardware_interface joint_name="j2"/>
    <xacro:armando_hardware_interface joint_name="j3"/>

  </ros2_control>
```

The `<hardware>` tags are used to signal to the ros2_control framework that the robot is being simulated throught Ignition Gazebo. The `armando_hardware_interface` macro defines the hardware interfaces of the joints. This macro has been included in the `armando_hardware_interface.xacro`, under the `/urdf` directory of the `armando_description` package.
The aforementioned xacro file defines the following interfaces for each joint:

```
<xacro:macro name="armando_hardware_interface" params="joint_name">

    <joint name="${joint_name}">
      <command_interface name="position"/>
      <state_interface name="position"/>
      <state_interface name="velocity"/>
    </joint>

</xacro:macro>
```

(c) Right after the `<ros2_control` tags, the `<gazebo>` tag has been added:

```
  <gazebo>
    <plugin filename="ign_ros2_control-system" name="ign_ros2_control::
    IgnitionROS2ControlPlugin">
    <parameters>$(find armando_description)/config/armando_controllers.yaml</
    parameters>
      <controller_manager_prefix_node_name>controller_manager</
    controller_manager_prefix_node_name>
    </plugin>
  </gazebo>
```

This code allows gazebo to do three things:

- read the hardware interfaces from the URDF;
- start the controller manager;
- spawn the controllers as described in the .yaml file.

The `armando_controllers.yaml` file (located in the `/config` directory, included in the `armando_description` package) contains the information about the controller manager and the controllers that will be loaded. At this point of development, the `armando_controllers.yaml` was defined so that the `position_controller` and the `joint_state_broadcaster` are spawned inside of the gazebo simulation:

```
controller_manager:
  ros__parameters:
    update_rate: 1000  # Hz
    use_sim_time: true
```

```
    position_controller:
      type: position_controllers/JointGroupPositionController

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

position_controller:
  ros__parameters:
    joints:
      - j0
      - j1
      - j2
      - j3

    state_publish_rate: 1000.0
    action_monitor_rate: 20.0
    allow_partial_joints_goal: true
    open_loop_control: true
    allow_integration_in_goal_trajectories: true
```

In order to allow the controllers to be loaded by the `armando_world.launch.py` file only after the gazebo simulation has started, the Node objects relative to the controllers were wrapped by the `RegisterEventHandler`. E.g., for the position controller:

```
delay_position_controller = RegisterEventHandler(
    event_handler=OnProcessExit(
        target_action=urdf_spawner_node,
        on_exit=[position_controller],
    )
)
```

The `target_action` parameter signals that the controller node will be started only after the `urdf_spawner_node` object (which is relative to the `create` node) has been started. The new `delay_position_controller` object will be passed to the LaunchDescription instead of the `position_controller` Node object. By launching the simulation and using the following command:

```
$ros2 run rqt_controller_manager rqt_controller_manager
```

the correct behaviour of the sensors and controller have been verified, as shown in figure 3 (the image shows all the controller available at the completion of the project).

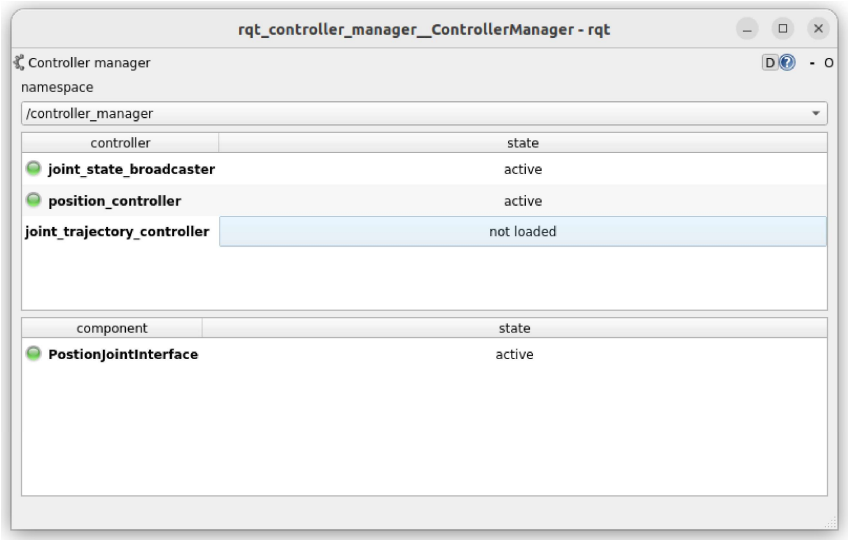

Figure 3: rqt_controller_manager interface

3. The third project specification requires the implementation of a camera sensor.

   (a) The `arm.urdf.xacro` file was updated in order to include a fixed joint whose parent is the base link and child is the new camera link. The camera joint was oriented in such a way that the framing of the camera is centered towards the center point of the range of motion of the joint j0. The camera link was then oriented so that it was facing the direction of the aforementioned camera. What has been descripted is shown in figure 4, where the camera is represented by the blue box.

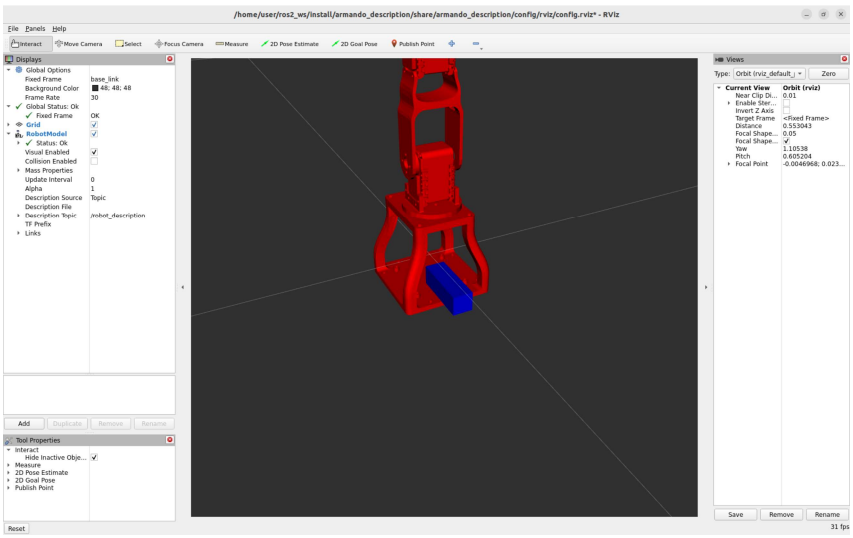

Figure 4: Camera link

   (b) The `armando_camera.xacro` was defined in the newly created `/urdf` folder in the `armando_gazebo` package (which was added to the CMakeList.txt). In the macro, most of the camera specifications have been parametrized. Finally, the macro has been included and called `arm.urdf.xacro`:

```
<xacro:include filename="$(find armando_gazebo)/urdf/armando_camera.xacro" />

<xacro:armando_camera camera_link="camera_link" camera_name="armando_camera"
  fov="1.047" width_res="320" height_res="240" near="0.1" far="100" topic_name
  ="camera"/>
```

The following code is the one relative to the `armando_camera.xacro` file:

```
<gazebo>
    <plugin filename="gz-sim-sensors-system"
    name="gz::sim::systems::Sensors">
    <render_engine>ogre2</render_engine>
  </plugin>
</gazebo>

<gazebo reference="${camera_link}">
  <sensor name="${camera_name}" type="camera">
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image>
        <width>${width_res}</width>
        <height>${height_res}</height>
      </image>
      <clip>
        <near>${near}</near>
        <far>${far}</far>
      </clip>
    </camera>
    <always_on>1</always_on>
    <update_rate>30</update_rate>
    <visualize>true</visualize>
    <topic>${topic_name}</topic>
  </sensor>
</gazebo>
```

(c) The project specifications required to use the `armando_gazebo.launch` to launch the simulation. Since this name was not previously mentioned, the choice was to implement under this name a launch file in the `/urdf` directory of the `armando_gazebo` package that launches the ros2-Ign bridge:

```
        def generate_launch_description():


    bridge_camera = Node(
        package='ros_ign_bridge',
        executable='parameter_bridge',
        arguments=[
            '/camera@sensor_msgs/msg/Image@gz.msgs.Image',
            '/camera_info@sensor_msgs/msg/CameraInfo@gz.msgs.CameraInfo',
            '--ros-args',
```

```
        '-r', '/camera:=/videocamera',
    ],
    output='screen'
)

return LaunchDescription([
    bridge_camera
])
```

The correct behaviour can be verified through the `rqt_image_view` tool, as shown in figure 5.

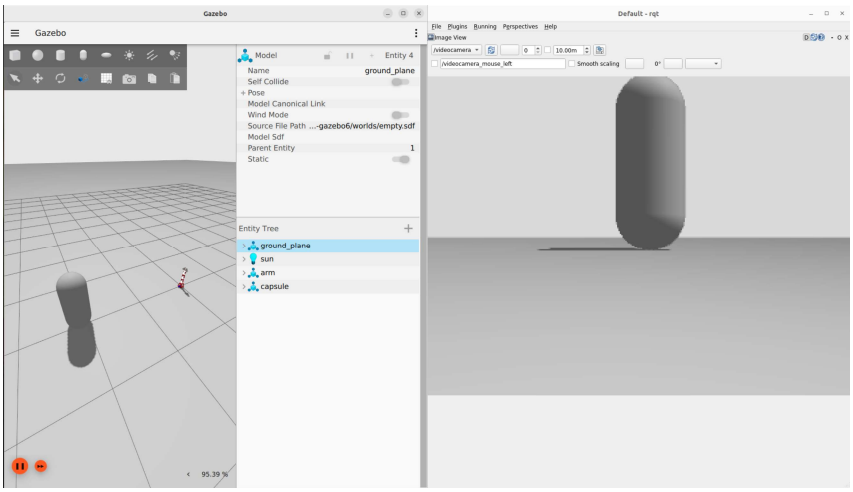

Figure 5: Gazebo environment and rqt_image_view side by side

4. The fourth project specification requires the implementation of a both publisher and subscriber node that should be able to send position or trajectory commands to the simulated Armando manipulator.

   (a) For the purpose of implementing the controllers a new package and node have been created:

```
ros2 pkg create --build-type ament_cmake --license Apache-2.0 --node-name arm_controller_node armando_controller
```

   By specifying the node name during the creation of the package, the `add_executable` in the `CMaskList.txt` is automatically updated to include the .cpp script relative to the node. The `ament_target_dependencies` command in the same file and the package manifest have been manually updated to include the dependencies requested by the project specification 4.a, which are:

   - `rlcpp`, the C++ client library for ROS2;
   - `sensor_msgs`, for the sensor-specific ROS2 messages;
   - `std_msgs`, for the primitive ROS2 messages.

   The following code is meant to specify the executable and dependencies in the `CMaskeList.txt` file:

```
add_executable(arm_controller_node src/arm_controller_node.cpp)
```

```
        ament_target_dependencies(arm_controller_node rclcpp sensor_msgs
std_msgs trajectory_msgs)
```

The following code is meant to specify the dependencies in the `package.xml` package manifest:

```
<depend>rclcpp</depend>
<depend>sensor_msgs</depend>
<depend>std_msgs</depend>
<depend>trajectory_msgs</depend>
```

(b) The following code, present in the `public` section of the `node` class, creates the node and starts the subscriber to the `joint_states`:

```
ArmControllerNode()
    : Node("arm_controller_node"), count_(0), current_pos{0.0, 0.0, 0.0, 0.0}
    {
      subscription_ = this->create_subscription<sensor_msgs::msg::JointState>(
      "joint_states", 10, std::bind(&ArmControllerNode::topic_callback, this, _1
    ));
    }
```

The callback function prints on terminal the current joint position received by the `joint_state` topic.

```
void topic_callback(const sensor_msgs::msg::JointState & msg)
    {
      RCLCPP_INFO(this->get_logger(), "\n J0 Pos: '%f'\n J1 Pos: '%f' \n J2 Pos:
     '%f'\n J3 Pos: '%f'\n", msg.position[0], msg.position[1], msg.position[2],
    msg.position[3]);
      for(int i=0; i<4;i++){
        current_pos[i] = msg.position[i];
      }
    }
```

(c) The `public` section of the `node` class has been updated to include a publisher on the `position_controller/commands` topic:

```
ArmControllerNode()
    : Node("arm_controller_node"), count_(0), current_pos{0.0, 0.0, 0.0, 0.0}
    {
      subscription_ = this->create_subscription<sensor_msgs::msg::JointState>(
      "joint_states", 10, std::bind(&ArmControllerNode::topic_callback, this, _1
    ));
      publisherpc_ = this->create_publisher<std_msgs::msg::Float64MultiArray>("
    position_controller/commands", 10);
      timer_ = this->create_wall_timer(
      1000ms, std::bind(&ArmControllerNode::timer_callback, this));
    }
```

The timer callback function for the `position_controller/commands` topic is as follows:

```
if(count_ <4){
    std_msgs::msg::Float64MultiArray message[4];
    message[0].data = {1.0, 0.0, 1.0, 0.0};
    message[1].data = {0.0, 1.0, 2.0, 1.0};
    message[2].data = {1.0, 0.0, 1.0, 2.0};
    message[3].data = {1.0, 1.0, 1.0, 1.0};

    //RCLCPP_INFO(this->get_logger(), "Publishing: '%f', '%f', '%f', '%f'",
    message[count_].data[0], message[count_].data[1], message[count_].data[2],
    message[count_].data[3]);
    publisherpc_->publish(message[count_]);


    bool reached_position = true;
    double epsilon = 1e-3;
    for(int i=0; i<4; i++){
    if(((current_pos[i] > message[count_].data[i] - epsilon)) && (current_pos[i]
     < (message[count_].data[i] + epsilon ))){
      reached_position = reached_position && true;
    }
    else{
      reached_position = reached_position && false;
    }
    }

    if(reached_position == true){
    RCLCPP_INFO(this->get_logger(), "POSITION %d REACHED", count_);
    count_++;
    }
    }
    else{
    RCLCPP_INFO(this->get_logger(), "ALL POSITIONS REACHED");
    }
```

`count` is a class-wide variable that is used to keep track of how many of the position goals have been reached. Each time the timer callback is called, the routine retrives the current position data from the `joint_states` topic subscriber, publishes the current position goal and checks if the current position in within a margin (defined by the `epsilon` variable) of the goal, and if it is, it increases the counter in order to set the next position goal.

(d) The `public` section of the `node` class has been updated to include a publisher on the `joint_trajectory_controller/joint_trajectory` topic and to declare the `controller_type` parameter:

```
ArmControllerNode()
    : Node("arm_controller_node"), count_(0), current_pos{0.0, 0.0, 0.0, 0.0}
    {
      subscription_ = this->create_subscription<sensor_msgs::msg::JointState>(
```

```
    "joint_states", 10, std::bind(&ArmControllerNode::topic_callback, this, _1
  ));
    publisherpc_ = this->create_publisher<std_msgs::msg::Float64MultiArray>("
  position_controller/commands", 10);
    publishertc_ = this->create_publisher<trajectory_msgs::msg::
  JointTrajectory>("joint_trajectory_controller/joint_trajectory", 10);
    timer_ = this->create_wall_timer(
    1000ms, std::bind(&ArmControllerNode::timer_callback, this));
    this->declare_parameter("controller_type",0);
  }
```

The `controller_type` parameter is used to select which of the two contoller routines to execute. From the command like the user is able to pass this argument.

```
    #to start the position controller publisher:
    $ ros2 run armando_controller arm_controller_node.cpp --ros-args -p
  controller_type:=0

    #to start the joint trajectory controller publisher:
    $ ros2 run armando_controller arm_controller_node.cpp --ros-args -p
  controller_type:=1
```

The argument is then parsed to discern which routine to use.
The position routine behaves in the same way as described in the previous point. The trajectory routine is the following:

```
trajectory_msgs::msg::JointTrajectory message_tc;
message_tc.joint_names={"j0", "j1", "j2", "j3"};
trajectory_msgs::msg::JointTrajectoryPoint point[4];

point[0].positions = {1.0, 0.0, 1.0, 0.0};
point[0].velocities = {0.0, 0.0, 0.0, 0.0};
point[0].time_from_start = rclcpp::Duration(15, 0);


point[1].positions = {0.0, 1.0, 1.0, 1.0};
point[1].velocities = {0.0, 0.0, 0.0, 0.0};
point[1].time_from_start = rclcpp::Duration(30, 0);


point[2].positions = {0.0, 0.0, 1.0, 2.0};
point[2].velocities = {0.0, 0.0, 0.0, 0.0};
point[2].time_from_start = rclcpp::Duration(45, 0);

point[3].positions = {1.0, 1.0, 1.0, 1.0};
point[3].velocities = {0.0, 0.0, 0.0, 0.0};
point[3].time_from_start = rclcpp::Duration(60, 0);

message_tc.points.push_back(point[0]);
```

```
message_tc.points.push_back(point[1]);
message_tc.points.push_back(point[2]);
message_tc.points.push_back(point[3]);
if(count_ < 1){
publishertc_->publish(message_tc);
count_++;
}
```

The code creates an array of joint-space trajectory points (defined by position and velocity) that is then published as a single message. The velocities in each point are set to 0.0 to ensure the passage through it by each joint. As an implementative choice, the duration value of each motion is relatively large (15 seconds), since the maximum velocity for each joint is 0.1 radiants (as defined in the URDF xacro file), and shorter times could not ensure the passage near each point.

In a similar way as the controller node, the launch file has been updated to be able to receive arguements through the command line in order to let the user choose which controller to load in gazebo.
The `controller_arg` is so defined:

```
controller_arg = DeclareLaunchArgument(
    'controller_type',
    default_value='0',
)
```

The `IfCondition()` function in passed as argument to the Node objects of the position controller and trajectory controller, so that they can be conditionally started based on the `controller_arg` value.

```
position_controller = Node( package="controller_manager",
                executable="spawner",
                arguments=["position_controller", "--controller-manager", "/
    controller_manager"],
                condition = IfCondition(PythonExpression([controller_type, " == 0
    "])))

joint_trajectory_controller = Node( package="controller_manager",
                executable="spawner",
                arguments=["joint_trajectory_controller", "--controller-manager",
     "/controller_manager"],
                condition = IfCondition(PythonExpression([controller_type, " == 1
    "])))
```

As a result, the following command line can be used to let the user decide which controller to load:

```
#to load the position controller:
$ ros2 launch armando_gazebo armando_world.launch.py controller_type:=0

#to load the trajectory controller:
$ ros2 launch armando_gazebo armando_world.launch.py controller_type:=1
```